Introduction to Robotics: ROS 2 Lab. 1.2

Containerization with Docker and Visual Studio Code

Professor: Pietro Falco
Tutor: Alessandro Adami, email: adamialess@dei.unipd.it

Department of Information Engineering (DEI)
Università degli Studi di Padova

March 6, 2026

1. Containerization with Docker

2. Visual Studio Code

Containerization with Docker

Setting up a robotics environment (Ubuntu + ROS 2 + Libraries) is often prone to errors across different operating systems.

- **Consistency**: "It works on my machine" applies to everyone.
- **Isolation**: Your lab environment won't mess up your personal OS.
- **Independency**: Run Ubuntu 22.04 on Windows, Mac, or even a newer Ubuntu version.

Key Terminology

- **Image**: A read-only template (the "blueprint").
- **Container**: A running instance of an image (the "living" environment).
- **Dockerfile**: A text script containing all commands to build an image.

A common point of confusion is the difference between an **Image** and a **Container**. Think of it as the difference between a recipe and a cake.

Docker Image (The Recipe)

- **Static**: It is a read-only snapshot.
- **Portable**: You can share it (via Docker Hub or GitHub).
- **Template**: It contains the OS (Ubuntu), ROS 2, and libraries, but it isn't "running."

Docker Container (The Cake)

- **Dynamic**: It is the living, breathing instance.
- **Ephemeral**: If you stop/delete it, any changes made *inside* (not in a volume) are lost.
- **Multiple**: You can bake 10 cakes from 1 recipe; you can run 10 containers from 1 image.

Robotics Context

The **Image** is your standardized lab environment. The **Container** is the specific "session" where you run your ROS 2 nodes.

Install Docker on your laptop:

- **Linux**: https://docs.docker.com/desktop/setup/install/linux/
- **Mac**: https://docs.docker.com/desktop/setup/install/mac-install/
- **Windows**: https://docs.docker.com/desktop/setup/install/windows-install/

Linux

It is suggested to work with Linux as OS, in case you already have it on your laptop, you can continue working on it without creating a Docker. You can also think about doing a dual boot on your laptop with other operative systems.

A `Dockerfile` automates the installation of Ubuntu and Python tools.

Example Dockerfile (Do not run this)

```
# 1. Use the official Ubuntu image
FROM ubuntu:22.04

# 2. Avoid prompts during installation
ENV DEBIAN_FRONTEND=noninteractive

# 3. Update and install Python & essential tools
RUN apt-get update && apt-get install -y \
    python3 \
    python3-pip \
    python3-numpy \
    git \
    && rm -rf /var/lib/apt/lists/*

# 4. Set the working directory
WORKDIR /home/lab_user
```

Best Practice

Always clean up apt caches (`rm -rf ...`) to keep your image size small!

Following the procedure to work with docker on linux is described.

```
docker --version #Check if Docker is installed in which version
sudo docker run hello-world # ---> Hello from Docker! ...
```

Then allow a user to work with Docker

```
sudo usermod -aG docker $USER
```

Log out completely and go back in from your user, then check:

```
docker ps
```

Install X11 utilities:

```
sudo apt update
sudo apt install -y x11-xserver-utils mesa-utils
```

Then verify:

```
glxinfo | grep "OpenGL renderer"
```

Allow Docker containers to use your display:

```
xhost +local:root
```

Your code must live outside Docker. This folder will be shared with the container.

```
mkdir -p ~/ros2_ws/src
```

Then create the Docker file:

- Create a directory:

```
mkdir ~/ros2_humble_docker
cd ~/ros2_humble_docker
```

- Create the Dockerfile:

```
nano Dockerfile
```

- Paste the docker file you find at this **GitHub folder**
- Still inside `~/ros2_humble_docker`:

```
docker build -t ros2-humble .
```

- Verify:

```
docker images | grep ros2-humble
docker rmi $ID #in case you want to remove a Docker with ID
```

Now the most important command:

```
docker run -it \
  --net=host \
  --env DISPLAY=$DISPLAY \
  --env QT_X11_NO_MITSHM=1 \
  --volume /tmp/.X11-unix:/tmp/.X11-unix:rw \
  --volume $HOME/ros2_ws:/root/ros2_ws \
  --name ros2_humble \
  ros2-humble
```

You should see:

```
root@your-machine:/root/ros2_ws#
```

> **Important**
>
> Inside the container, you are now a **root user** in a pure Ubuntu 22.04 environment. You can run `python3`

Now you are inside the container, and ROS 2 is already sourced. Now, test the container:

```
xeyes    # test GUI
ros2     # test ROS 2
rviz2    # test Rviz
gazebo   # test Gazebo
ros2 launch gazebo_ros gazebo.launch.py   #test Gazebo launch
```

Run inside the container

```
apt update
apt install -y \
  gazebo \
  ros-humble-gazebo-ros-pkgs \
  ros-humble-gazebo-ros2-control
```

And then source ROS 2 again:

```
source /opt/ros/humble/setup.bash
```

Run inside the container

```
cd ~/ros2_ws/src
ros2 pkg create my_test_pkg --build-type ament_cmake
cd ~/ros2_ws
colcon build
source install/setup.bash
```

When we build a new workspace, a new `setup.bash` file is created, and we need to source this if we want to use it.

Colcon

Colcon is your building tool

Source

Always `source` after building and after opening a new terminal

Common Commands

Command	Description
`docker ps -a`	List all containers (running and stopped).
`docker images`	List all built images on your disk.
`docker start [Name]`	Start a container.
`docker start -ai [Name]`	Start and Attach a container.
`docker exec -it [Name] bash (or eventually sh)`	Exec Into a Running container.
`docker ps`	List all containers (running).
`xhost +local:root`	Access control list.
`docker stop [ID]`	Stop a running container from another terminal.
`exit`	Stop a running container from inside the container.
`docker rm [ID]`	Delete a container.
`docker rmi [Name]`	Delete an image.

Cleanup

Use `docker system prune` to remove all unused data and free up disk space!

A ROS2 workspace (usually named `ros2_ws`) is organized into four main directories:

- `src/`: The **only** place where you write code. It contains your packages.
- `build/`: Where intermediate files are stored during compilation.
- `install/`: Where your executable nodes and libraries are installed.
- `log/`: Contains logging information about the build process.

Golden Rule

Never manually edit files in `build`, `install`, or `log`. If things get messy, you can safely delete these three folders and rebuild from `src`.

How code moves through your workspace:

- **Developer Input**: Write code in `src/`.
- **The Action**: Run `colcon build`.
- **The Result**: Executables appear in `install/`.

$$\texttt{src/} \xrightarrow{\text{colcon build}} \texttt{build/ \& install/}$$

Common Error

If you see `Package not found`, it's 99% likely you forgot to run `source install/setup.bash` in that specific terminal window.

Visual Studio Code

To develop efficiently, we will use **Visual Studio Code** combined with the **Remote Development** extension pack.

- **Integrated Environment**: Edit files locally while they are running inside Docker.
- **IntelliSense**: Get Python code completion and error checking inside the container.
- **Integrated Terminal**: Open an Ubuntu terminal directly inside your editor.

Required Extensions

Search for and install the following in VSCode:

1. **Remote - Containers** (part of the Remote Development pack).
2. **Python** (by Microsoft).
3. **Docker** (by Microsoft).

Once your container is running in your terminal, follow these steps to start coding:

1. **Start the Container**: Ensure your Docker container is running.
2. **Select Command**: Choose **"Attach to Running Container..."** from the list (with keyboard Ctrl + Shift + p).
3. **Pick your Lab Container**: Select `lab_container` (or the name you gave it).

What happens next?

VS Code will open a new window. Look at the bottom-left corner: it should now say **Container ros2-humble**. You are now "inside" Ubuntu!

Working in a container is slightly different from working locally:

- **Opening Folders**: Use Ctrl + o and navigate to your `WORKDIR` (e.g., `/home/lab_user/code`).
- **Extensions**: Some extensions may be **installed again** inside the container. VS Code will show a button: "Install in Container".
- **Terminal**: Press Ctrl + ' to open a terminal; it will automatically be an Ubuntu bash shell inside your container.

Tip for ROS 2

When we start using ROS 2, the VS Code Python extension will help you find the correct libraries for autocomplete if you select the correct interpreter inside the container.

When you "Attach" VS Code to a container, you are creating a bridge between your Windows/Mac/Linux UI and the Ubuntu environment inside Docker.

The Host (Your Laptop)

- **VS Code UI**: The editor, theme, and window management.
- **Source Code**: Stored in your `ros2_ws` folder (Local Disk).
- **X-Server**: Handles the windows for Rviz/Gazebo.

The Container (The Lab)

- **VS Code Server**: A small agent running "inside" to handle IntelliSense.
- **ROS 2 Stack**: All the libraries, compilers, and nodes.
- **The App**: Your Python code actually executes here.

The "Magic" of Volumes

The `-v` flag links a folder on your **Host** to a folder in the **Container**.
If you save a file in VS Code, it is updated in both places simultaneously!

Developing in ROS 2 requires specific environment variables (e.g., `PYTHONPATH`) that are only set **inside** the container.

- **IntelliSense**: VS Code reads the libraries installed in `/opt/ros/humble/`. Without the Remote extension, your editor would show "Module not found" errors for `rclpy`.
- **Unified Debugging**: You can set breakpoints inside your Python nodes and step through code while the robot is "running" in Docker.
- **No Setup Leakage**: You don't need to install Python 3.10 or ROS 2 on your Windows/Mac host; everything stays inside the container "box."

Key Shortcut

Ctrl + Shift + P → **Remote-Containers: Attach to Running Container** is your gateway to the development environment.

This is how you will work in every session:

Step-by-Step Cycle

Step	Tool	Action
1	`Terminal`	Run `docker run -it -v ...` to start the lab env.
2	`VS Code`	❯ Attach to Running Container.
3	`VS Code`	Write your Python nodes (`.py` files).
4	`VS Code`	Run code in the integrated terminal: `python3 my_script.py`.
5	`Terminal`	Type `exit` to stop the container (files stay safe in the volume).

- **Volume Mapping**: Ensures your code survives container deletion.
- **Isolation**: Keeps your Windows/Mac OS clean and stable.